

VOLUME 0 — PROJECT FOUNDATION

CHAPTER 0.2

Project Constitution

Binding Rules for the Entire Project Lifecycle

Document Title	Project Constitution
Chapter Reference	0.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Precedence	This document overrides any conflicting instruction in later volumes unless formally amended under Section 9.

This Constitution defines the non-negotiable rules that govern how this project is documented, decided, and built. Every later volume and chapter must comply with it. If a later chapter appears to conflict with this one, this Constitution wins until it is formally amended (Section 9).

1. Project Rules

- One Volume = One Goal. Every volume in this documentation set exists to answer exactly one question about the product (e.g. "what are we building," "how does it look," "how is it engineered").
- One Chapter = One Document. Each chapter is delivered as its own standalone file, so it can be reviewed, approved, and referenced independently.
- Sequential approval. The next volume does not start until the current volume's chapters are complete and explicitly approved by the project owner.
- No implementation before Volume 6. No production code is written until Volumes 0 through 6 (Foundation → Mobile App Architecture) are documented and approved. Volumes 7–12 may be produced in parallel with early implementation.
- Every decision is written down. A choice that is not recorded in these documents does not count as decided — it will be re-litigated the next time it comes up.
- Documentation drives code, not the other way around. Code is an expression of the approved documents, not a substitute for them.

2. Technology Constraints

Framework: Flutter / Dart, single codebase targeting Android and iOS. This is fixed and not open for reconsideration without a formal change request (Section 9).

Primary development machine: Windows. The goal is 90–100% of development — UI, business logic, chunking, upload engine, local persistence, testing — to happen on Windows using Android as the primary test target.

iOS builds: Since no Mac is available locally, iOS building/signing/testing is handled via a cloud Mac / CI build service (e.g. Codemagic, Ionic Appflow, or a rented Mac-in-the-cloud), documented in full in Volume 7 — Development Environment. iOS work is secondary to Android during the MVP phase.

Cloud storage: AWS S3 is the system of record for uploaded video chunks. All chunk uploads are final-destination S3, using resumable/multipart upload.

Backend services: Authentication, push notifications, and remote configuration may use Firebase (Auth, FCM, Remote Config) alongside a custom REST/GraphQL API for Projects, Tasks, and Sessions — the exact split is finalized in Volume 4 (Backend Architecture) and recorded as an ADR in Chapter 3.2.

Local persistence: A local embedded database (finalized in Volume 6.5) plus the device filesystem for raw video chunks — the app must function fully offline for recording and queue uploads for later.

Video tooling: The platform camera plugin for capture, and FFmpeg (or platform-native equivalents) for local chunking/processing, as detailed in Volume 5 — Recording Engine.

No unapproved dependencies: Any new third-party package or service must be recorded against the relevant Volume 3/4/6 chapter before being added to the project.

3. Architecture Principles

- Offline-first. Every core workflow (viewing tasks, recording, chunking, queueing uploads) must work with no network connection; the network is only required to sync, never to unblock the collector.
- Never lose a take. Once the collector taps Stop, the raw footage must be safely written to local storage before any other operation (chunking, upload) is attempted.
- Recording and uploading are decoupled. A collector must always be able to start a new recording while a previous session is still uploading.
- Idempotent, resumable uploads. Re-attempting an upload (after a crash, a network drop, or an app restart) must never produce duplicate or corrupted chunks in S3.
- Clean separation of concerns. The codebase is layered into data, domain, and presentation layers (finalized in Volume 3 — Technical Architecture); business logic does not live inside UI widgets.
- Single source of truth for state. Any given piece of app state (auth session, recording session, upload queue) has exactly one owner in the architecture — no duplicate, conflicting state stores.
- Configuration over hard-coding. Capture parameters (chunk duration, resolution, bitrate) are designed from day one to be remotely configurable (Volume 3.3 and Chapter 11), even if remote config ships in a later phase.
- Secure by default. Tokens and credentials live in secure storage (Volume 6.7), all network calls use TLS, and local video files are treated as sensitive data until confirmed uploaded.
- Design for testability. Modules are structured so recording, chunking, and upload logic can be unit- and integration-tested independently of the UI (Volume 9).

4. Coding Standards

- Follow the official Effective Dart style guide as the baseline for all Dart/Flutter code.
- Null safety is mandatory; no use of legacy nullable-unsafe patterns.
- Static analysis (the Dart/Flutter linter) must run clean before any code is considered done — lint rule set is finalized in Volume 3.7.

- No magic numbers or hard-coded strings for anything configurable (chunk length, endpoints, thresholds) — these live in a constants/config layer.
- Every public class, method, and non-trivial function carries a documentation comment explaining intent, not just mechanics.
- Errors are handled explicitly at every layer boundary — no silently swallowed exceptions, especially around recording and upload (Volume 6.9 — Error Handling).
- Files are kept to a reasonable, single-responsibility size; a file that is trying to do too many things is split, not extended indefinitely.
- All new logic is accompanied by tests per the strategy defined in Volume 9.

5. Naming Conventions

Dart files	snake_case.dart (e.g. recording_controller.dart)
Dart classes / enums	PascalCase (e.g. UploadQueueManager)
Dart variables / functions	camelCase (e.g. startChunkUpload())
Constants	lowerCamelCase or SCREAMING_CAPS per Effective Dart, applied consistently within a file
Git branches	type/short-description, e.g. feature/chunk-upload-retry, fix/login-token-refresh
Commit messages	Conventional Commits — feat:, fix:, docs:, refactor:, test:, chore:
API endpoints	kebab-case, resource-oriented, e.g. /projects/{id}/tasks
Database tables / columns	snake_case, singular table names avoided in favor of plural (e.g. sessions, chunks)
S3 object keys / chunk files	Deterministic, collision-proof pattern finalized in Volume 5.14 — File Naming Strategy (must encode project, task, session, and chunk index)
Documentation files	Chapter_<volume>.<chapter>_<Title>.docx, matching this document's own naming

6. Decision-Making Rules

- Any decision with more than one reasonable technical option (framework choice, storage engine, state management library, cloud provider) is recorded as an Architecture Decision Record in Chapter 3.2, including the options considered and why one was chosen.
- Once a chapter is approved, its decisions are binding until formally revisited through Section 9 (Change Management Rules) — no silent overrides in later chapters or in code.
- Where the project owner has not yet expressed a preference, Claude proposes a recommendation with tradeoffs rather than leaving the decision open, and flags it clearly as "proposed, pending approval."
- Conflicting requirements between chapters are resolved in favor of whichever chapter was approved earliest, and the conflict is logged for correction.

7. AI Collaboration Rules

These rules govern how Claude (or any AI collaborator) works on this project across sessions, since the documentation itself will be produced primarily through AI-assisted sessions.

- Respect the volume order. Claude does not generate Volume N+1 content until the user has confirmed Volume N is approved.
- One chapter at a time. Within a volume, Claude completes and delivers one chapter document before starting the next, unless the user explicitly asks for multiple in parallel.
- No code before Volume 6 is approved. Claude does not scaffold or write implementation code until the foundation, product, design, architecture, backend, recording engine, and mobile architecture volumes are signed off.
- Always check prior chapters. Before drafting new content, Claude treats previously approved chapters as binding context and does not contradict them without flagging the conflict.
- Never fabricate requirements. Where information is missing, Claude states its assumption explicitly in the document (e.g. an "Assumptions" callout) rather than inventing unstated business requirements.
- Deliver as the agreed format. Each chapter is produced as a standalone, properly formatted Word document (.docx) unless the user requests a different format for that chapter.
- Flag risk and ambiguity. If a request would create inconsistency with an already-approved chapter, Claude says so before proceeding rather than silently complying.
- Preserve the user's own words for vision and business intent. Where the user has articulated a goal or workflow directly, Claude structures and formats it rather than replacing the substance.

8. Source of Truth Policy

- This documentation set — Volumes 0 through 12 — is the single source of truth for the product, in the version most recently approved by the project owner.
- If code and documentation ever diverge, the documentation governs, and the code is corrected — unless a change request (Section 9) has been raised and approved to update the documentation instead.
- Every document carries a version number and status (Draft / Approved / Superseded) in its Document Control block, so the current source of truth is always identifiable at a glance.
- Superseded documents are retained (not deleted) for history, and clearly marked as superseded with a pointer to the replacement.

9. Change Management Rules

Because the documents are the source of truth, changing them is deliberate, not casual. Any change to an already-approved chapter follows this process:

1. Propose — the change and its reason are described in plain language (what is changing and why it's needed now).
2. Impact assessment — Claude identifies every other chapter/volume that references or depends on the thing being changed.
3. Approval — the project owner explicitly approves the change before any document is edited.
4. Version bump — the affected chapter's version number is incremented and its Document Control block updated (e.g. 1.0 → 1.1).
5. Changelog entry — the change is recorded in Chapter 11.5 — Changelog, including the old and new state and the reason.

End of Chapter 0.2 — Project Constitution